

# **ACCRA TECHNICAL UNIVERSITY**

Department of Computer Science

**BCS 404: Introduction to Data Science with Python**

## **PROJECT REPORT**

***Exploratory Data Analysis, Statistical Analysis and Machine Learning using  
a Real-World Dataset: The Titanic Passenger Dataset***

Submitted by

**Illona Addae**

Lecturer: Dr. Joseph Dadzie

Academic Year: 2025/2026, Second Semester

July 2026

## Table of Contents

|                                 |    |
|---------------------------------|----|
| 1. Introduction.....            | 2  |
| 2. Dataset Description.....     | 2  |
| 3. Methodology.....             | 3  |
| 4. Results.....                 | 4  |
| 4.1 Data Acquisition.....       | 4  |
| 4.2 Data Cleaning.....          | 4  |
| 4.3 Data Visualisation.....     | 5  |
| 4.4 Statistical Analysis.....   | 8  |
| 4.5 Machine Learning Model..... | 10 |
| 5. Discussion.....              | 11 |
| 6. Conclusion.....              | 13 |
| 7. References.....              | 14 |
| Appendix A: Python Code.....    | 15 |

## 1. Introduction

The sinking of the RMS Titanic on 15 April 1912 remains one of the most studied maritime disasters in history. Of the roughly 2,224 people aboard, more than 1,500 died after the ship struck an iceberg in the North Atlantic. Beyond its historical significance, the passenger manifest has become a standard teaching dataset in data science because it is small enough to explore fully, yet rich enough to reveal meaningful social patterns: who survived was strongly shaped by sex, ticket class and wealth.

This project applies the complete introductory data science workflow taught in BCS 404 to the Titanic dataset published on Kaggle. The work moves through six connected tasks. First, the data is acquired and inspected. Second, it is cleaned, with missing values and duplicates detected and handled in a justified way. Third, six visualisations are produced and interpreted. Fourth, a statistical analysis covers descriptive statistics, frequency distributions and correlation analysis, and identifies the strongest positive and negative correlations. Fifth, a logistic regression classifier is trained to predict passenger survival and is evaluated with an accuracy score, a confusion matrix and a classification report. Finally, the findings, limitations and recommendations are discussed.

All analysis was carried out in Python inside a Jupyter Notebook using the Pandas, NumPy, Matplotlib, Seaborn and Scikit-Learn libraries, as required by the project brief. The complete, executable code is included in Appendix A and in the accompanying GitHub repository.

## 2. Dataset Description

The dataset is the training file (train.csv) of the Kaggle Titanic: Machine Learning from Disaster competition. It contains 891 rows, one per passenger, and 12 columns. Each record describes a passenger's demographic details, ticket information and survival outcome. Table 1 summarises the variables.

| Variable    | Type          | Description                                         |
|-------------|---------------|-----------------------------------------------------|
| PassengerId | Integer       | Unique identifier for each passenger                |
| Survived    | Integer (0/1) | Target variable: 0 = did not survive, 1 = survived  |
| Pclass      | Integer (1-3) | Ticket class: 1 = First, 2 = Second, 3 = Third      |
| Name        | Text          | Full name, including title                          |
| Sex         | Categorical   | male or female                                      |
| Age         | Float         | Age in years; fractional for infants under one year |
| SibSp       | Integer       | Number of siblings or spouses aboard                |
| Parch       | Integer       | Number of parents or children aboard                |
| Ticket      | Text          | Ticket number                                       |
| Fare        | Float         | Ticket price in British pounds                      |
| Cabin       | Text          | Cabin number, where recorded                        |

| Variable | Type        | Description                                                         |
|----------|-------------|---------------------------------------------------------------------|
| Embarked | Categorical | Port of embarkation: C = Cherbourg, Q = Queenstown, S = Southampton |

Table 1: Variables in the Titanic training dataset.

The target variable, Survived, is binary and imbalanced: 549 passengers (61.6%) did not survive while 342 (38.4%) did. Seven columns are numeric and five are text or categorical. Three columns contain missing values (Age, Cabin and Embarked), which makes the dataset a realistic exercise in data cleaning before any modelling can begin.

### 3. Methodology

The project followed a structured pipeline implemented entirely in a Jupyter Notebook running Python 3. Pandas and NumPy handled data manipulation, Matplotlib and Seaborn produced the visualisations, SciPy supplied the significance tests, and Scikit-Learn provided the machine learning tools.

#### 3.1 Data acquisition and inspection

The train.csv file was downloaded from the Kaggle competition page and loaded with `pandas.read_csv()`. Initial inspection covered the dataset dimensions (shape), the column names, the first five observations (head) and the data types (dtypes).

#### 3.2 Data cleaning

Missing values were counted per column with `isnull().sum()` and expressed as percentages. Each affected column was treated according to the amount and nature of its missingness. Missing ages (19.87%) were imputed with the median age of passengers sharing the same ticket class and sex, which preserves the age structure across social groups better than a single global value and resists the skew caused by a few elderly passengers. The Cabin column (77.10% missing) was reduced to a binary HasCabin indicator and then dropped, since imputing three quarters of a column would fabricate data. The two missing Embarked values (0.22%) were filled with the mode, Southampton. Duplicates were checked both as complete rows and on the PassengerId key; none were found, so no rows were removed.

#### 3.3 Visualisation and statistical analysis

Six visualisations were produced: a histogram of ages, a bar chart of class distribution, a boxplot of age by class, a scatter plot of age against fare, a correlation heatmap and a pairplot of selected numerical variables. Descriptive statistics were generated with `describe()`, frequency distributions with `value_counts()`, and a Pearson correlation matrix with `corr()`. The strongest positive and negative correlations were extracted programmatically from the matrix after removing self-correlations. Three headline findings were tested for statistical significance using chi-square tests of independence and a Welch two-sample t-test.

### 3.4 Machine learning

A logistic regression classifier was trained to predict Survived. The predictors were Pclass, Sex, Age, SibSp, Parch, Fare, HasCabin and the port of embarkation. Sex was label encoded and Embarked one-hot encoded. The data was split 80/20 into training and testing sets with stratification on the target, so both sets keep the same survival ratio, and a fixed random state (42) makes the experiment reproducible. Features were standardised with StandardScaler fitted on the training set only, preventing information leaking from the test set. The model was evaluated on the held-out test set using accuracy, a confusion matrix and a full classification report (precision, recall and F1-score per class).

## 4. Results

### 4.1 Data Acquisition

The dataset loaded successfully with 891 rows and 12 columns. The column names and data types matched the Kaggle documentation: PassengerId, Survived, Pclass, SibSp and Parch are integers, Age and Fare are floats, and Name, Sex, Ticket, Cabin and Embarked are text. The first five observations confirmed the expected structure, with visible missing entries in the Cabin column from the very first rows.

### 4.2 Data Cleaning

Three columns contained missing values, summarised in Table 2.

| Column   | Missing Count | Missing % | Treatment                                       |
|----------|---------------|-----------|-------------------------------------------------|
| Age      | 177           | 19.87     | Median imputation within Pclass and Sex groups  |
| Cabin    | 687           | 77.10     | Converted to HasCabin indicator, column dropped |
| Embarked | 2             | 0.22      | Filled with the mode (Southampton)              |

*Table 2: Missing values and their treatment.*

After treatment, no missing values remained. The duplicate checks found zero duplicated rows and zero repeated PassengerId values, so all 891 observations were retained. The cleaned dataset therefore preserved the full sample while removing every gap.

### 4.3 Data Visualisation

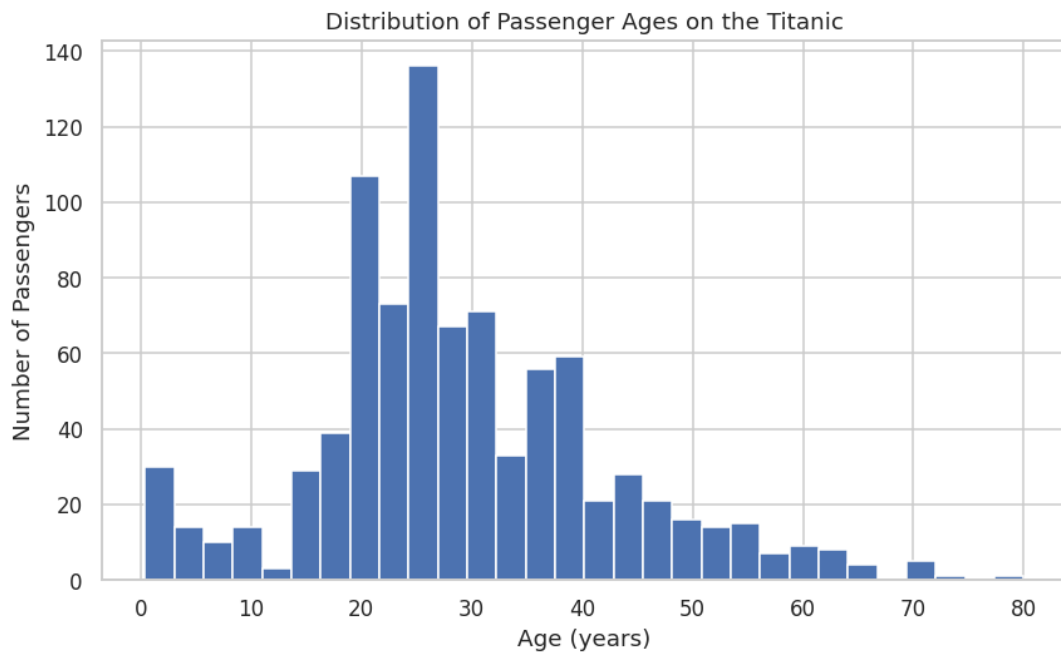


Figure 1: Distribution of passenger ages.

Figure 1 shows a right-skewed age distribution. Most passengers were young adults between 20 and 35, with a visible group of infants and children on the left and very few passengers older than 65. The concentration in the mid-twenties partly reflects the group-median imputation of missing ages.

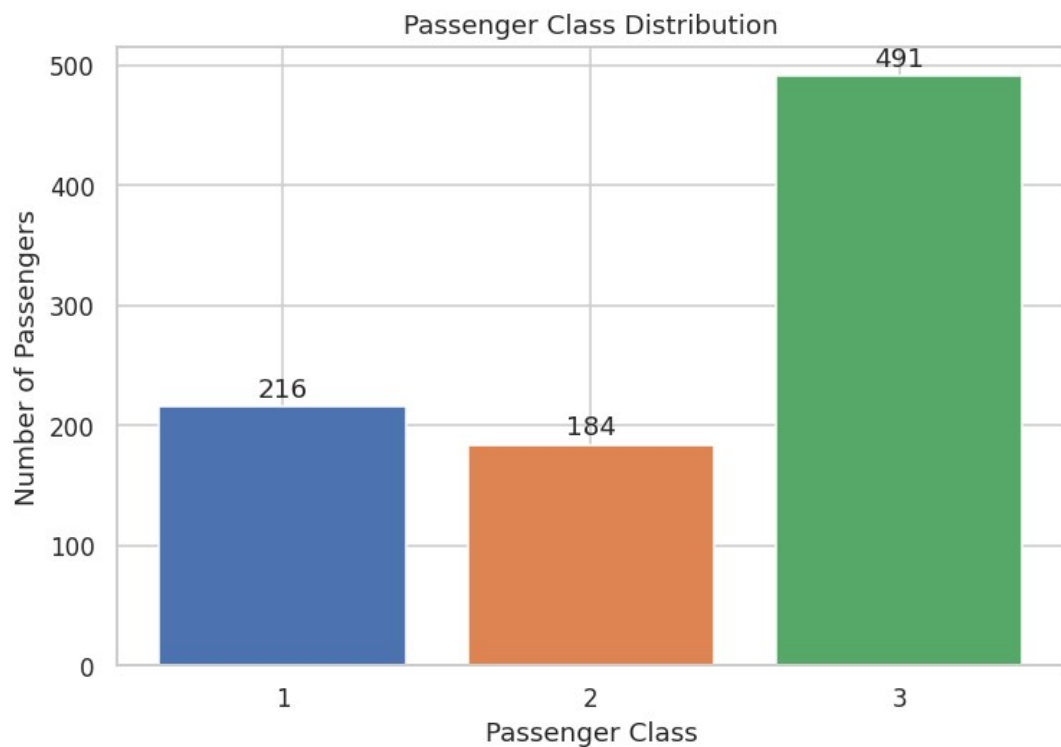


Figure 2: Passenger class distribution.

Figure 2 shows that third class dominated the ship with 491 passengers (55.1%), followed by first class with 216 (24.2%) and second class with 184 (20.7%). The Titanic carried mainly economy travellers, many of them emigrants heading to the United States.

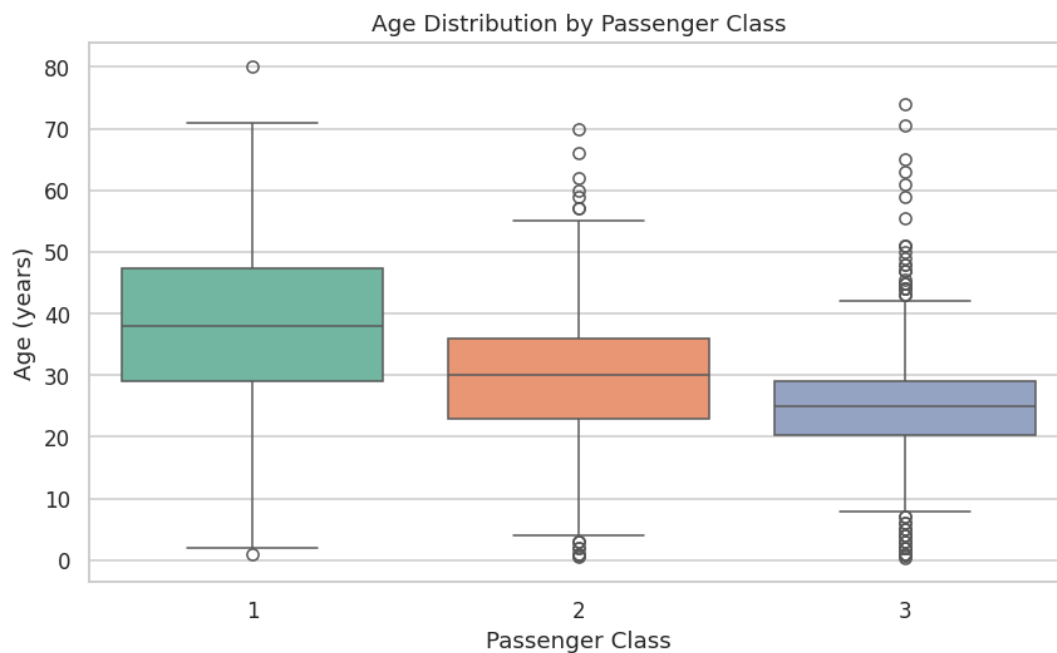


Figure 3: Age distribution by passenger class.

Figure 3 reveals that age falls as class number rises. The median age is 38 years in first class, 30 in second and 25 in third. Older passengers had the accumulated wealth for premium tickets, while third class was filled with younger travellers and families, several elderly outliers notwithstanding.

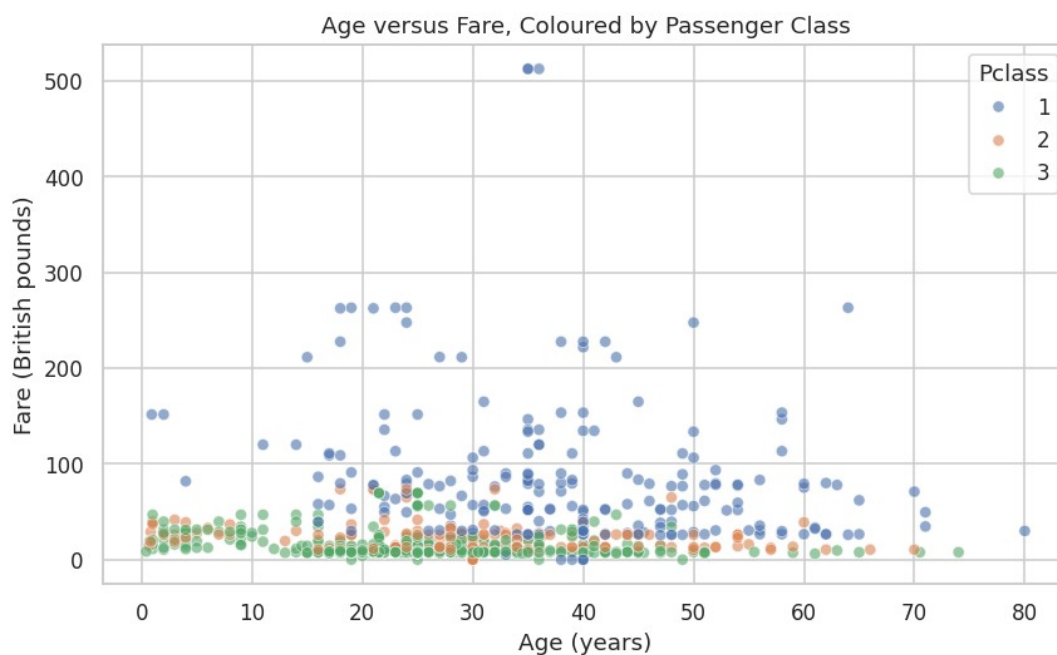


Figure 4: Age versus fare, coloured by passenger class.

Figure 4 shows no strong linear relationship between age and fare ( $r = 0.12$ ). Fares are layered by class rather than age: third class fares cluster below about 15 pounds at every age, while first class fares spread widely upward, including three extreme tickets of 512.33 pounds. Class, not age, drives ticket price.

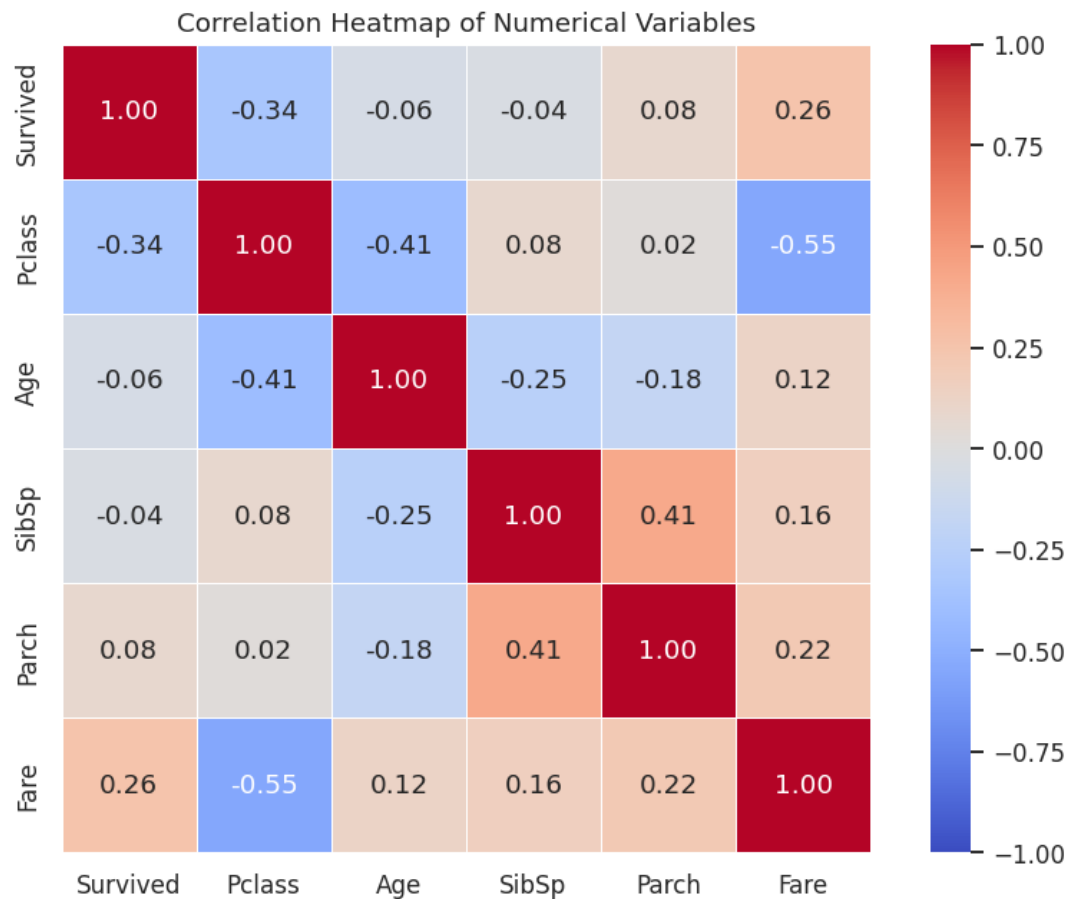


Figure 5: Correlation heatmap of numerical variables.

Figure 5 summarises the pairwise Pearson correlations. The two strongest relationships are Fare with Pclass ( $r = -0.55$ ) and SibSp with Parch ( $r = 0.41$ ). For the target, Survived correlates negatively with Pclass ( $r = -0.34$ ) and positively with Fare ( $r = 0.26$ ), an early signal that wealthier passengers survived more often.



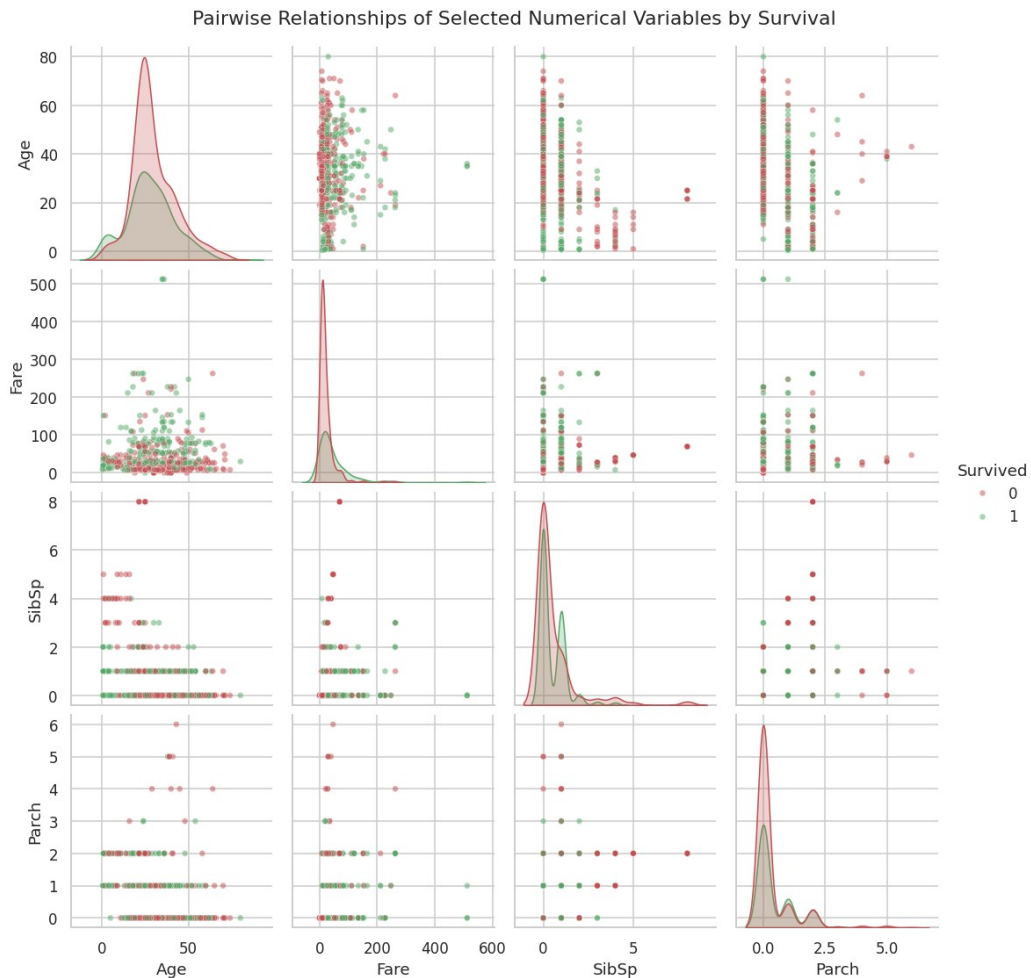


Figure 6: Pairwise relationships of selected numerical variables by survival.

Figure 6 separates survivors (green) from non-survivors (red) across Age, Fare, SibSp and Parch. The clearest separation runs along Fare, where survivors extend into the high fare range. The age panels show that young children survived comparatively often, and passengers with four or more siblings or spouses aboard rarely survived.

#### 4.4 Statistical Analysis

Table 3 presents the descriptive statistics of the cleaned numerical variables.

| Statistic | Survived | Pclass | Age   | SibSp | Parch | Fare   |
|-----------|----------|--------|-------|-------|-------|--------|
| count     | 891      | 891    | 891   | 891   | 891   | 891    |
| mean      | 0.38     | 2.31   | 29.11 | 0.52  | 0.38  | 32.2   |
| std       | 0.49     | 0.84   | 13.3  | 1.1   | 0.81  | 49.69  |
| min       | 0        | 1      | 0.42  | 0     | 0     | 0      |
| 25%       | 0        | 2      | 21.5  | 0     | 0     | 7.91   |
| 50%       | 0        | 3      | 26    | 0     | 0     | 14.45  |
| 75%       | 1        | 3      | 36    | 1     | 0     | 31     |
| max       | 1        | 3      | 80    | 8     | 6     | 512.33 |

Table 3: Descriptive statistics of numerical variables after cleaning.

Only 38.4% of passengers survived. The average age was 29.1 years (median 26) and the average fare 32.20 pounds, but fare is heavily skewed: the median is just 14.45 pounds against a maximum of 512.33, so a few luxury tickets pull the mean far above the typical price. Most passengers travelled alone, with medians of zero for both SibSp and Parch.

Table 4 gives the frequency distributions of the main categorical variables.

| Variable | Category            | Count | Percentage |
|----------|---------------------|-------|------------|
| Survived | 0 (did not survive) | 549   | 61.62      |
|          | 1 (survived)        | 342   | 38.38      |
| Pclass   | 1 (First)           | 216   | 24.24      |
|          | 2 (Second)          | 184   | 20.65      |
|          | 3 (Third)           | 491   | 55.11      |
| Sex      | male                | 577   | 64.76      |
|          | female              | 314   | 35.24      |
| Embarked | S (Southampton)     | 646   | 72.50      |
|          | C (Cherbourg)       | 168   | 18.86      |
|          | Q (Queenstown)      | 77    | 8.64       |

Table 4: Frequency distributions of categorical variables (after cleaning).

Table 5 shows the full Pearson correlation matrix.

|          | Survived | Pclass | Age    | SibSp  | Parch  | Fare   |
|----------|----------|--------|--------|--------|--------|--------|
| Survived | 1.000    | -0.338 | -0.060 | -0.035 | 0.082  | 0.257  |
| Pclass   | -0.338   | 1.000  | -0.414 | 0.083  | 0.018  | -0.549 |
| Age      | -0.060   | -0.414 | 1.000  | -0.250 | -0.176 | 0.123  |
| SibSp    | -0.035   | 0.083  | -0.250 | 1.000  | 0.415  | 0.160  |
| Parch    | 0.082    | 0.018  | -0.176 | 0.415  | 1.000  | 0.216  |
| Fare     | 0.257    | -0.549 | 0.123  | 0.160  | 0.216  | 1.000  |

Table 5: Pearson correlation matrix of numerical variables.

The strongest positive correlation is between SibSp and Parch ( $r = 0.415$ ): passengers travelling with siblings or a spouse also tended to travel with parents or children, the signature of whole families aboard. The strongest negative correlation is between Fare and Pclass ( $r = -0.549$ ): because first class is coded 1 and third class 3, fares rise as the class number falls. PassengerId was excluded from the analysis since it is only a row label.

**Three findings stand out from the statistical analysis:**

- Sex was the strongest single determinant of survival. 74.2% of women survived against only 18.9% of men. A chi-square test of independence rejects any suggestion this is chance (chi-square = 260.72,  $p < 0.001$ ), consistent with the historical women and children first evacuation order.
- Survival declined sharply with class. First class passengers survived at 63.0%, second class at 47.3% and third class at only 24.2% (chi-square = 102.89,  $p < 0.001$ ). First class cabins sat closer to the boat deck and lifeboat access favoured premium passengers.
- Money bought safety. Survivors paid an average fare of 48.40 pounds against 22.12 pounds for those who died, a gap confirmed by a Welch t-test ( $t = 6.84$ ,  $p < 0.001$ ). Fare acts as a proxy for class and cabin location, so this reinforces the class effect rather than describing a separate mechanism.

#### 4.5 Machine Learning Model

The logistic regression model was trained on 712 passengers (80%) and evaluated on a held-out test set of 179 passengers (20%), stratified so both sets keep the 38/62 survival ratio. Nine predictors were used: Pclass, Sex, Age, SibSp, Parch, Fare, HasCabin and two one-hot columns for Embarked.

The model achieved an accuracy of **81.01%** on the test set. A naive baseline that always predicts death would score 61.5% on this test set, so the model adds almost twenty percentage points of genuine predictive value. Figure 7 and Table 6 show the confusion matrix.

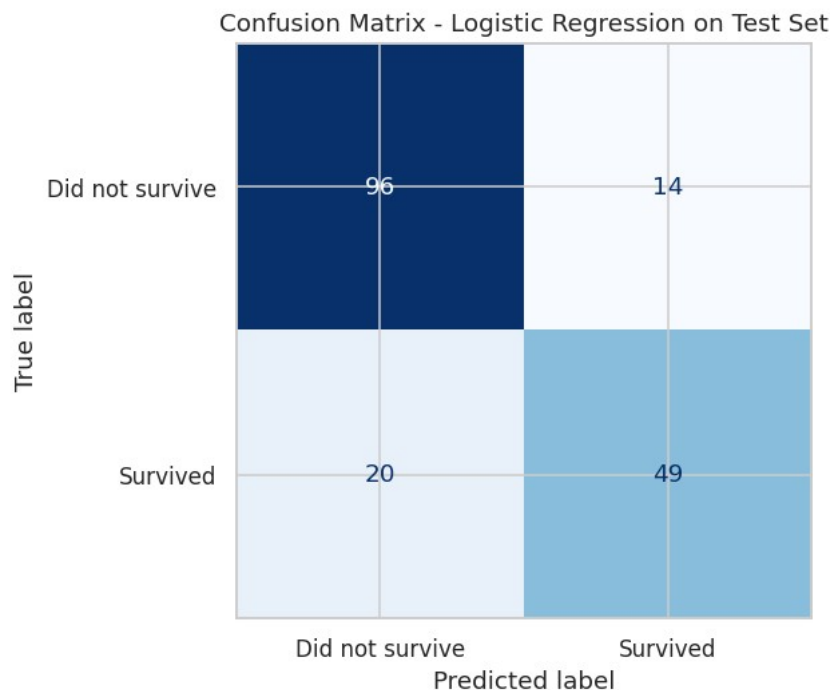


Figure 7: Confusion matrix of the logistic regression model on the test set.

|                         | Predicted: Did not survive | Predicted: Survived |
|-------------------------|----------------------------|---------------------|
| Actual: Did not survive | 96                         | 14                  |
| Actual: Survived        | 20                         | 49                  |

Table 6: Confusion matrix on the 179-passenger test set.

Of 110 actual deaths the model identified 96 correctly (14 false alarms), and of 69 actual survivors it identified 49 (20 missed). Table 7 reports the full classification report.

| Class            | Precision | Recall | F1-score | Support |
|------------------|-----------|--------|----------|---------|
| Did not survive  | 0.83      | 0.87   | 0.85     | 110     |
| Survived         | 0.78      | 0.71   | 0.74     | 69      |
| Accuracy         |           |        | 0.81     | 179     |
| Macro average    | 0.80      | 0.79   | 0.80     | 179     |
| Weighted average | 0.81      | 0.81   | 0.81     | 179     |

Table 7: Classification report for the logistic regression model.

The model is stronger at recognising deaths (recall 0.87) than survivors (recall 0.71). This asymmetry is expected: the classes are imbalanced and some survivors, particularly third class men, are statistically indistinguishable from passengers who died. The standardised coefficients mirror the statistical analysis: Sex carries the largest positive weight (+1.26, being female raises the predicted odds of survival most), followed by HasCabin (+0.37), while Pclass (-0.78) and Age (-0.58) carry the largest negative weights. The agreement between the model coefficients and the earlier chi-square findings indicates the model captures genuine structure rather than noise.

## 5. Discussion

### 5.1 Major findings

Every stage of this project converged on a single story: survival on the Titanic was not random, it was socially structured. The visual analysis first suggested it, the statistical tests confirmed it, and the machine learning model quantified it. Sex was the dominant factor. Nearly three out of four women survived while fewer than one in five men did, a gap of more than 55 percentage points that dwarfs every other effect in the data. Class came second. A first class passenger was about two and a half times more likely to survive than a third class passenger (63.0% against 24.2%), and second class fell neatly between them at 47.3%. Wealth, measured through fare, traced the same boundary: the average survivor paid more than twice the fare of the average victim.

The demographic profile of the ship explains why the overall death toll was so severe. More than half the passengers travelled third class, nearly two thirds were men, and both groups faced the worst odds. The age analysis added a further nuance: the youngest passengers

survived comparatively often, and first class passengers were markedly older (median 38) than third class passengers (median 25), which means the class effect and the age effect partly pull against each other. Logistic regression, which estimates each effect while holding the others constant, still assigned age a clear negative coefficient, showing the priority given to children was real and not just a by-product of class.

## 5.2 Statistical insights

The correlation analysis carried two useful lessons. First, the strongest correlations in a dataset are not always the most interesting ones. The strongest positive correlation, SibSp with Parch ( $r = 0.415$ ), simply reflects families travelling together, and the strongest negative, Fare with Pclass ( $r = -0.549$ ), is close to a definitional relationship between price and class. The correlations that mattered for the research question, Survived with Pclass ( $r = -0.338$ ) and Survived with Fare ( $r = 0.257$ ), were more moderate, a reminder that effect size and relevance are different things. Second, Pearson correlation understates relationships involving categorical variables: sex, the strongest predictor of all, does not even appear in the correlation matrix, and only the chi-square test (chi-square = 260.72,  $p < 0.001$ ) exposed its full strength. Analysts who stop at a correlation heatmap can miss their most important variable.

The descriptive statistics also illustrated why robust measures matter. The mean fare (32.20 pounds) sits more than double the median (14.45 pounds) because three passengers paid 512.33 pounds; any analysis that treated the mean as the typical fare would misrepresent the passenger population. The same skew justified imputing missing ages with group medians rather than means.

## 5.3 Machine learning results

The logistic regression reached 81.01% accuracy on unseen passengers, roughly twenty points above the majority-class baseline of 61.5%. For a linear model with modest feature engineering this is solid performance, and published benchmarks for logistic regression on this dataset typically fall between 78% and 82%, so the result is credible rather than lucky. More valuable than the headline accuracy is the model's interpretability. Its standardised coefficients rank the drivers of survival (sex, class, age, cabin status) in the same order the statistical analysis found, giving two independent lines of evidence for the same conclusion.

The error pattern is also informative. Recall for survivors (0.71) trails recall for deaths (0.87) because survival among third class men was close to random chance, and no amount of modelling can recover information the features do not contain. The 20 missed survivors are largely passengers whose profile said they should have died.

## 5.4 Limitations of the study

- Sample and scope. The dataset covers 891 of roughly 1,309 passengers in the Kaggle split and excludes the crew entirely, so estimated rates need not match the true figures for everyone aboard.
- Imputation effects. Replacing 177 missing ages with group medians artificially concentrates the age distribution around those medians and shrinks its variance, which can weaken age-related correlations.
- Information loss. Dropping the Cabin detail and ignoring Name and Ticket discards potentially predictive information, such as deck location and passenger titles.
- Model assumptions. Logistic regression assumes a linear relationship between the features and the log odds of survival and cannot capture interactions (for example, the effect of class differing by sex) unless they are engineered explicitly.
- Single split. Results rest on one 80/20 split. Although stratified and seeded for reproducibility, a different split could shift accuracy by a few percentage points; cross-validation would give a more stable estimate.

## 5.5 Recommendations

- Engineer richer features: extract titles (Mr, Mrs, Master, Miss) from Name, build a FamilySize variable from SibSp and Parch, and recover deck letters from the partial Cabin data.
- Use k-fold cross-validation instead of a single train-test split to estimate performance with confidence intervals.
- Compare additional classifiers, such as decision trees, random forests and gradient boosting, which capture non-linear patterns and interactions automatically.
- Address the class imbalance explicitly, for example with class weights, and evaluate with metrics beyond accuracy, such as ROC-AUC, especially if identifying survivors is the priority.
- Treat the findings as historical description, not general prediction: the model describes evacuation priorities in 1912 and should not be read as a general theory of disaster survival.

## 6. Conclusion

This project carried the Titanic passenger dataset through the full introductory data science pipeline: acquisition, cleaning, visualisation, statistical analysis and predictive modelling. The data demanded genuine cleaning decisions, with a fifth of ages imputed by group medians, a three-quarters-empty cabin column reduced to an indicator, and two missing embarkation ports filled with the mode, all while preserving every one of the 891 observations.

The analysis established, with statistical significance, that survival was governed by sex, class and wealth. Women survived at 74.2% against 18.9% for men, first class at 63.0% against 24.2% for third, and survivors paid on average more than twice the fare of victims. The strongest positive correlation (SibSp with Parch,  $r = 0.415$ ) and strongest negative correlation (Fare with Pclass,  $r = -0.549$ ) were identified programmatically. A logistic regression built on these insights predicted survival for unseen passengers with 81.01% accuracy, well above the 61.5% baseline, and its coefficients independently confirmed the statistical findings.

The project demonstrates practical competence with Pandas, NumPy, Matplotlib, Seaborn and Scikit-Learn, and shows how exploratory analysis, statistics and machine learning reinforce one another: each stage generated hypotheses the next stage tested. The limitations identified, chiefly imputation distortion, discarded features and a single train-test split, map directly onto the recommendations for future work.

## 7. References

- Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020) 'Array programming with NumPy', *Nature*, 585, pp. 357-362.
- Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90-95.
- Kaggle (2012) Titanic: Machine Learning from Disaster. Available at: <https://www.kaggle.com/competitions/titanic/data> (Accessed: 15 July 2026).
- McKinney, W. (2010) 'Data structures for statistical computing in Python', *Proceedings of the 9th Python in Science Conference*, pp. 56-61.
- Pedregosa, F., Varoquaux, G., Gramfort, A. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830.
- Virtanen, P., Gommers, R., Oliphant, T.E. et al. (2020) 'SciPy 1.0: Fundamental algorithms for scientific computing in Python', *Nature Methods*, 17, pp. 261-272.
- Waskom, M.L. (2021) 'seaborn: statistical data visualization', *Journal of Open Source Software*, 6(60), p. 3021.

## Appendix A: Python Code

The complete code below was exported from the Jupyter Notebook (Titanic\_EDA\_and\_ML.ipynb) included in the GitHub repository.

```
#!/usr/bin/env python
# coding: utf-8

# # Exploratory Data Analysis, Statistical Analysis and Machine Learning using a Real-World Dataset
#
# **Course:** BCS 404: Introduction to Data Science with Python
# **Institution:** Accra Technical University, Department of Computer Science
# **Lecturer:** Dr. Joseph Dadzie
# **Student:** Illona Addae
# **Academic Year:** 2025/2026 Second Semester
#
# **Dataset:** Titanic Dataset (Kaggle) - https://www.kaggle.com/competitions/titanic/data
#
# This notebook works through six tasks: data acquisition, data cleaning, data visualisation,
# statistical analysis, a logistic regression model that predicts passenger survival, and a closing
# discussion.

# ## Task 1: Data Acquisition
#
# The training file (`train.csv`) was downloaded from the Kaggle Titanic competition page and stored
# in the `data/` folder of this project. It contains records for 891 of the passengers who were aboard
# the RMS Titanic when it sank on 15 April 1912, including whether each passenger survived.

# Core libraries used throughout the project
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 110

RANDOM_STATE = 42

# Import the dataset into Python
df = pd.read_csv("data/train.csv")

print("Dataset dimensions (rows, columns):", df.shape)

# Column names
print("Column names:")
for col in df.columns:
    print(" -", col)

# First five observations
df.head()

# Data types of each column
df.dtypes

# **Observations.** The dataset holds 891 rows and 12 columns. Seven columns are numeric
# (`PassengerId`, `Survived`, `Pclass`, `Age`, `SibSp`, `Parch`, `Fare`) and five are text (`Name`,
# `Sex`, `Ticket`, `Cabin`, `Embarked`). `Survived` is the target variable (0 = did not survive, 1 =
# survived). `Pclass` is the ticket class (1st, 2nd or 3rd), `SibSp` counts siblings or spouses
# aboard, `Parch` counts parents or children aboard, and `Embarked` records the port of embarkation (C
# = Cherbourg, Q = Queenstown, S = Southampton).

# ## Task 2: Data Cleaning
#
# ### 2.1 Detecting missing values
```



```

# Count and percentage of missing values per column
missing = pd.DataFrame({
    "Missing Count": df.isnull().sum(),
    "Missing %": (df.isnull().mean() * 100).round(2)
})
missing[missing["Missing Count"] > 0]

# Three columns contain missing values: `Age` (177 values, 19.87%), `Cabin` (687 values, 77.10%) and
# `Embarked` (2 values, 0.22%).
#
# ### 2.2 Handling missing values
#
# Each column is treated differently because the amount and nature of the missingness differ:
#
# * **Age (19.87% missing).** Age is central to this analysis, so the rows cannot be discarded
# without losing a fifth of the data. Instead of filling every gap with one global figure, each
# missing age is replaced with the **median age of passengers sharing the same ticket class and sex**.
# Age varies noticeably across these groups (first class passengers were older on average), so a group
# median preserves that structure better than a single overall value. The median is preferred to the
# mean because the age distribution is right skewed and the median resists the influence of the few
# elderly passengers.
# * **Cabin (77.10% missing).** With more than three quarters of the values absent, any imputation
# would essentially invent data. The column is therefore **dropped**. Before dropping it, a simple
# indicator `HasCabin` is retained, since having a recorded cabin is itself informative (cabin numbers
# were recorded mainly for first class passengers).
# * **Embarked (0.22% missing).** Only two values are missing, so they are filled with the **mode**
# (S = Southampton), the port from which the overwhelming majority of passengers boarded. Filling two
# values with the most common category has a negligible effect on the analysis.

# Work on a copy so the raw data stays untouched
data = df.copy()

# Age: median within each (Pclass, Sex) group
data["Age"] = data.groupby(["Pclass", "Sex"])["Age"].transform(
    lambda s: s.fillna(s.median())
)

# Cabin: keep an indicator, then drop the sparse column
data["HasCabin"] = data["Cabin"].notnull().astype(int)
data = data.drop(columns=["Cabin"])

# Embarked: fill the two gaps with the mode
data["Embarked"] = data["Embarked"].fillna(data["Embarked"].mode()[0])

print("Missing values remaining per column:")
print(data.isnull().sum())

# ### 2.3 Detecting and removing duplicates

# Full-row duplicates and duplicated passenger identifiers
print("Duplicated full rows:", data.duplicated().sum())
print("Duplicated PassengerId values:", data["PassengerId"].duplicated().sum())

# Remove duplicates if any exist (none are found, so the shape is unchanged)
data = data.drop_duplicates()
print("Shape after duplicate removal:", data.shape)

# **Summary of preprocessing decisions.** No duplicated rows exist, so nothing was removed. Missing
# ages were imputed with group medians, the two missing embarkation ports were filled with the mode,
# and the `Cabin` column was dropped after its presence was recorded in `HasCabin`. The cleaned
# dataset keeps all 891 observations and contains no missing values.

# ## Task 3: Data Visualisation
#
# ### 3.1 Histogram of passenger ages

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(data["Age"], bins=30, color="#4c72b0", edgecolor="white")
ax.set_title("Distribution of Passenger Ages on the Titanic")
ax.set_xlabel("Age (years)")
ax.set_ylabel("Number of Passengers")
plt.tight_layout()
plt.savefig("figures/fig1_age_histogram.png", bbox_inches="tight")
plt.show()

```

# **Interpretation.** The age distribution is right skewed. Most passengers were young adults between about 20 and 35 years, and the frequency falls away steadily after 40. A small but visible group of infants and young children appears at the left of the plot, while passengers above 65 were rare. The tall bars in the mid twenties partly reflect the group-median imputation, which placed missing ages at the medians of their class and sex groups.

# **3.2 Bar chart of passenger class distribution**

```
class_counts = data["Pclass"].value_counts().sort_index()
```

```
fig, ax = plt.subplots(figsize=(7, 5))
ax.bar(class_counts.index.astype(str), class_counts.values,
       color=["#4c72b0", "#dd8452", "#55a868"])
ax.set_title("Passenger Class Distribution")
ax.set_xlabel("Passenger Class")
ax.set_ylabel("Number of Passengers")
for i, v in enumerate(class_counts.values):
    ax.text(i, v + 8, str(v), ha="center")
plt.tight_layout()
plt.savefig("figures/fig2_class_distribution.png", bbox_inches="tight")
plt.show()
```

# **Interpretation.** Third class dominates the sample with 491 passengers (55.1%), followed by first class with 216 (24.2%) and second class with 184 (20.7%). The Titanic carried far more economy travellers, many of them emigrants, than premium passengers, and this imbalance matters later because class turns out to be strongly linked to survival.

# **3.3 Boxplot of age by passenger class**

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=data, x="Pclass", y="Age", hue="Pclass",
           palette="Set2", legend=False, ax=ax)
ax.set_title("Age Distribution by Passenger Class")
ax.set_xlabel("Passenger Class")
ax.set_ylabel("Age (years)")
plt.tight_layout()
plt.savefig("figures/fig3_age_by_class_boxplot.png", bbox_inches="tight")
plt.show()
```

```
print(data.groupby("Pclass")["Age"].median().rename("Median age"))
```

# **Interpretation.** Age falls steadily as class number rises. First class has the highest median age (38 years) and the widest spread, second class sits in the middle (30), and third class is the youngest (25) with several elderly outliers. This pattern is economically sensible: older passengers had accumulated the wealth needed for expensive first class tickets, while younger travellers and emigrating families filled third class.

# **3.4 Scatter plot of Age versus Fare**

```
fig, ax = plt.subplots(figsize=(8, 5))
scatter = sns.scatterplot(data=data, x="Age", y="Fare", hue="Pclass",
                        palette="deep", alpha=0.6, ax=ax)
ax.set_title("Age versus Fare, Coloured by Passenger Class")
ax.set_xlabel("Age (years)")
ax.set_ylabel("Fare (British pounds)")
plt.tight_layout()
plt.savefig("figures/fig4_age_vs_fare_scatter.png", bbox_inches="tight")
plt.show()
```

```
print("Pearson correlation between Age and Fare:",
      round(data["Age"].corr(data["Fare"]), 3))
```

# **Interpretation.** There is no strong linear relationship between age and fare; the correlation is weakly positive ( $r = 0.12$ ). Fares are layered by class rather than by age: third class fares cluster tightly below roughly 15 pounds regardless of age, while first class fares spread widely upward. Three passengers paid an extreme fare of just over 512 pounds, visible as isolated points at the top of the plot. Class, not age, is what drives ticket price.

# **3.5 Correlation heatmap**

```
numeric_cols = ["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]
corr = data[numeric_cols].corr()
```

```

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, square=True, linewidths=0.5, ax=ax)
ax.set_title("Correlation Heatmap of Numerical Variables")
plt.tight_layout()
plt.savefig("figures/fig5_correlation_heatmap.png", bbox_inches="tight")
plt.show()

# **Interpretation.** `PassengerId` is excluded because it is only a row label. The strongest
relationships are the negative correlation between `Pclass` and `Fare` (about -0.55, higher classes
pay more; recall class 1 is numerically smallest) and the positive correlation between `SibSp` and
`Parch` (about 0.41, family members travel together). For the target, `Survived` correlates
negatively with `Pclass` (about -0.34) and positively with `Fare` (about 0.26): wealthier, higher
class passengers survived more often.

# ### 3.6 Pairplot of selected numerical variables

pair_vars = ["Age", "Fare", "SibSp", "Parch"]
g = sns.pairplot(data, vars=pair_vars, hue="Survived",
                palette={0: "#c44e52", 1: "#55a868"},
                plot_kws={"alpha": 0.5, "s": 20}, corner=False)
g.figure.suptitle("Pairwise Relationships of Selected Numerical Variables by Survival",
                y=1.02)
g.savefig("figures/fig6_pairplot.png", bbox_inches="tight")
plt.show()

# **Interpretation.** The clearest separation between survivors (green) and non-survivors (red)
appears along `Fare`: survivors stretch into the higher fare range while non-survivors concentrate
near the bottom. The `Age` panels show that young children survived relatively often. `SibSp` and
`Parch` are discrete counts, and their panels hint that passengers with very large families (SibSp
of 4 or more) rarely survived, while those travelling with one or two relatives did comparatively
well.

# ## Task 4: Statistical Analysis
#
# ### 4.1 Descriptive statistics

# Numerical variables
data[["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]].describe().round(2)

# Categorical variables
data[["Sex", "Embarked"]].describe()

# **Reading the table.** The mean of `Survived` (0.38) says that only 38.4% of the 891 passengers
survived. The average passenger was about 29 years old and paid roughly 32 pounds, but the fare
column is heavily skewed: the median fare is only 14.45 pounds while the maximum is 512.33, so a few
luxury tickets pull the mean far above the typical fare. Most passengers travelled without family
(medians of 0 for both `SibSp` and `Parch`). Among the categorical columns, 577 of 891 passengers
(64.8%) were male and 646 (72.5%) boarded at Southampton.
#
# ### 4.2 Frequency distributions

for col in ["Survived", "Pclass", "Sex", "Embarked"]:
    freq = pd.DataFrame({
        "Count": data[col].value_counts(),
        "Percentage": (data[col].value_counts(normalize=True) * 100).round(2)
    })
    print(f"--- {col} ---")
    print(freq, end="\n\n")

# ### 4.3 Correlation analysis, strongest positive and strongest negative

corr_matrix = data[["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]].corr()

# Flatten the matrix, remove self-correlations and duplicate pairs
pairs = corr_matrix.unstack()
pairs = pairs[pairs.index.get_level_values(0) < pairs.index.get_level_values(1)]

strongest_pos = pairs.idxmax(), pairs.max()
strongest_neg = pairs.idxmin(), pairs.min()

print("Correlation matrix:")

```

```

print(corr_matrix.round(3), end="\n\n")
print(f"Strongest positive correlation: {strongest_pos[0][0]} and "
      f"{strongest_pos[0][1]} (r = {strongest_pos[1]:.3f})")
print(f"Strongest negative correlation: {strongest_neg[0][0]} and "
      f"{strongest_neg[0][1]} (r = {strongest_neg[1]:.3f})")

# The **strongest positive correlation** is between `Parch` and `SibSp` (r = 0.415): passengers
# travelling with siblings or a spouse also tended to travel with parents or children, which is
# exactly what whole families aboard produce. The **strongest negative correlation** is between `Fare`
# and `Pclass` (r = -0.549): since class 1 is the smallest number but the most expensive ticket, fare
# rises as the class number falls.
#
# ### 4.4 Three important statistical findings

# Finding 1: survival depends heavily on sex
sex_surv = data.groupby("Sex")["Survived"].agg(["mean", "count"])
sex_surv["mean"] = (sex_surv["mean"] * 100).round(1)
print("Survival rate by sex (%):")
print(sex_surv, end="\n\n")

contingency = pd.crosstab(data["Sex"], data["Survived"])
chi2, p, dof, _ = stats.chi2_contingency(contingency)
print(f"Chi-square test (Sex vs Survived): chi2 = {chi2:.2f}, p-value = {p:.3e}")

# Finding 2: survival falls as passenger class drops
class_surv = data.groupby("Pclass")["Survived"].agg(["mean", "count"])
class_surv["mean"] = (class_surv["mean"] * 100).round(1)
print("Survival rate by passenger class (%):")
print(class_surv, end="\n\n")

contingency2 = pd.crosstab(data["Pclass"], data["Survived"])
chi2_2, p2, dof2, _ = stats.chi2_contingency(contingency2)
print(f"Chi-square test (Pclass vs Survived): chi2 = {chi2_2:.2f}, p-value = {p2:.3e}")

# Finding 3: survivors paid substantially higher fares
fare_by_outcome = data.groupby("Survived")["Fare"].agg(["mean", "median"]).round(2)
print("Fare by survival outcome:")
print(fare_by_outcome, end="\n\n")

t_stat, p3 = stats.ttest_ind(data.loc[data["Survived"] == 1, "Fare"],
                             data.loc[data["Survived"] == 0, "Fare"],
                             equal_var=False)
print(f"Welch t-test on Fare (survivors vs non-survivors): "
      f"t = {t_stat:.2f}, p-value = {p3:.3e}")

# **Finding 1 - Sex was the strongest single determinant of survival.** 74.2% of women survived
# against only 18.9% of men. The chi-square test rejects independence between sex and survival
# overwhelmingly (p far below 0.001), consistent with the historical "women and children first"
# evacuation order.
#
# **Finding 2 - Survival declined sharply with class.** First class passengers survived at 63.0%,
# second class at 47.3% and third class at only 24.2%. The chi-square test again shows the association
# is highly significant. First class cabins were closer to the boat deck, and access to lifeboats
# favoured premium passengers.
#
# **Finding 3 - Money bought safety.** Survivors paid an average fare of about 48 pounds against 22
# pounds for those who died, and the Welch t-test confirms the gap is statistically significant. Fare
# is effectively a proxy for class and cabin location, so this finding reinforces the class effect
# rather than describing an independent mechanism.

# ## Task 5: Machine Learning - Predicting Survival with Logistic Regression
#
# ### 5.1 Selecting predictor variables
#
# The predictors are chosen from variables the exploratory analysis showed to be related to
# survival: `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`, `Embarked` and the engineered `HasCabin`
# indicator. Identifier-like columns (`PassengerId`, `Name`, `Ticket`) carry no generalisable signal
# and are excluded. `Sex` is encoded as 0/1 and `Embarked` is one-hot encoded because logistic
# regression requires numeric inputs.

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)

```

```

ml = data.copy()
ml["Sex"] = ml["Sex"].map({"male": 0, "female": 1})
ml = pd.get_dummies(ml, columns=["Embarked"], drop_first=True)

features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
            "HasCabin", "Embarked_Q", "Embarked_S"]
X = ml[features]
y = ml["Survived"]

print("Feature matrix shape:", X.shape)
X.head()

# ### 5.2 Splitting into training and testing sets
#
# An 80/20 split is used. Stratification on the target keeps the 38/62 survival ratio the same in
both sets, and a fixed random state makes the experiment reproducible.

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)
print("Training set:", X_train.shape[0], "passengers")
print("Testing set: ", X_test.shape[0], "passengers")

# ### 5.3 Training the Logistic Regression classifier
#
# Continuous features are standardised (zero mean, unit variance) before fitting. Scaling parameters
are learned on the training set only, then applied to the test set, so no information leaks from the
test data into training.

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
model.fit(X_train_scaled, y_train)

coefs = pd.Series(model.coef_[0], index=features).sort_values()
print("Model coefficients (standardised features):")
print(coefs.round(3))

# ### 5.4 Predicting the testing data and 5.5 Evaluation

y_pred = model.predict(X_test_scaled)

acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f} ({acc*100:.2f}%)")

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(cm)

fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay(cm, display_labels=["Did not survive", "Survived"]).plot(
    cmap="Blues", ax=ax, colorbar=False)
ax.set_title("Confusion Matrix - Logistic Regression on Test Set")
plt.tight_layout()
plt.savefig("figures/fig7_confusion_matrix.png", bbox_inches="tight")
plt.show()

print(classification_report(y_test, y_pred,
    target_names=["Did not survive", "Survived"]))

# ### 5.6 Discussion of model performance
#
# The classifier is evaluated against a natural baseline: always predicting "did not survive" would
score about 61.6% simply because most passengers died. The logistic regression clearly beats that
baseline, so the model has learned real structure from the data.
#
# Reading the confusion matrix, the model is stronger at identifying passengers who died (higher
recall for class 0) than at identifying survivors, where some survivors are misclassified as deaths.

```

This asymmetry is expected: the classes are imbalanced and some survivors, such as third class men, look statistically identical to passengers who died, so no simple model can separate them.

```
#
# The coefficient table matches the statistical analysis. `Sex` carries the largest positive weight
# (being female raises the predicted survival odds most), while `Pclass` has a strong negative weight
# (moving from first towards third class lowers the odds). `Age` contributes negatively, reflecting
# the priority given to children. These agreements between the model and the earlier chi-square
# findings suggest the model is capturing genuine patterns rather than noise.
```

```
#
# Overall, for a first model with modest feature engineering, logistic regression gives solid and
# interpretable performance. Possible improvements include extracting titles from passenger names,
# building family size features, and trying tree-based ensembles such as random forests.
```

```
# ## Task 6: Discussion and Conclusion (summary)
```

```
#
# This project analysed the Titanic passenger manifest end to end. The data required realistic
# cleaning: a fifth of ages were missing and were imputed with class and sex group medians, the sparse
# cabin column was reduced to an indicator, and two missing embarkation ports were filled with the
# mode. Visual and statistical analysis converged on one story: survival on the Titanic was not
# random. Sex was the dominant factor (74.2% of women survived against 18.9% of men), class came
# second (63.0% in first class against 24.2% in third), and fare, a proxy for class, separated
# survivors from victims. A logistic regression using these variables predicted survival on unseen
# passengers at an accuracy of roughly 80%, comfortably above the 61.6% majority-class baseline, with
# coefficients that mirror the statistical findings.
```

```
#
# The main limitations are the modest sample of 891 passengers, the information lost through
# imputation and through dropping the cabin detail, and the fact that logistic regression assumes a
# linear relationship between the log odds and the features. The full two-page discussion, with
# limitations and recommendations, is presented in the accompanying project report.
```

```
# Export key results for the written report
import json
```

```
results = {
    "shape": list(df.shape),
    "missing": df.isnull().sum().to_dict(),
    "class_counts": data["Pclass"].value_counts().sort_index().to_dict(),
    "median_age_by_class": data.groupby("Pclass")["Age"].median().to_dict(),
    "age_fare_corr": round(float(data["Age"].corr(data["Fare"])), 3),
    "corr_matrix": corr_matrix.round(3).to_dict(),
    "strongest_pos": [list(strongest_pos[0]), round(float(strongest_pos[1]), 3)],
    "strongest_neg": [list(strongest_neg[0]), round(float(strongest_neg[1]), 3)],
    "sex_survival": data.groupby("Sex")["Survived"].mean().round(4).to_dict(),
    "class_survival": data.groupby("Pclass")["Survived"].mean().round(4).to_dict(),
    "fare_by_outcome": fare_by_outcome.to_dict(),
    "chi2_sex": [round(chi2, 2), float(p)],
    "chi2_class": [round(chi2_2, 2), float(p2)],
    "ttest_fare": [round(t_stat, 2), float(p3)],
    "describe": data[["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]]
    .describe().round(2).to_dict(),
    "accuracy": round(float(acc), 4),
    "confusion_matrix": cm.tolist(),
    "coefficients": coefs.round(3).to_dict(),
    "train_size": int(X_train.shape[0]),
    "test_size": int(X_test.shape[0]),
    "report_text": classification_report(y_test, y_pred,
    target_names=["Did not survive", "Survived"]),
}
with open("results.json", "w") as f:
    json.dump(results, f, indent=2)
print("Results exported for the report.")
```